

EdgeIM: An Efficient Edge-based Process Model Discovery Technique

Xuan Su¹, Cong Liu^{2,3}, Faming Lu^{1*}, Long Cheng⁴, Qingtian Zeng¹, Shouli Zhang⁵

¹School of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, China

²School of Computer Science and Technology, Shandong University of Technology, Zibo, China

³NOVA Information Management School, Nova University of Lisbon, Lisbon, Portugal

⁴School of Control and Computer Engineering, North China Electric Power University, Beijing, China

⁵College of Information Science and Engineering, Shandong Agricultural University, Taian, China

{su_xuan2021, liuconchina, fm_lu, qtzeng, zhangshouli}@163.com, lcheng@ncepu.edu.cn

Abstract—The rapid expansion of Internet of Things (IoT) devices has led to an explosion of event data, posing significant challenges for traditional process model discovery techniques in terms of scalability and discovery accuracy. These techniques rely on centralized storage and processing, which are hindered by data transfer limitations, storage capacity, and computational overhead in distributed IoT environments. Edge-based model discovery techniques offer a promising solution for analyzing large-scale IoT data. However, existing techniques suffer from low efficiency and an inability to handle complex process structures. To address these challenges, we propose EdgeIM, an efficient edge-based process model discovery technique that enhances efficiency and model accuracy. EdgeIM operates in three key stages: preprocessing and feature-preserving sampling to eliminate redundant data, local processing at edge nodes to extract key structural features, and global feature aggregation at a central node for model discovery. EdgeIM has been implemented on the open-source process mining platform PM4Py, and experimental results on nine public event logs demonstrate that, compared to existing edge-based model discovery techniques, EdgeIM significantly improves discovery efficiency while maintaining high model quality.

Index Terms—Process Mining, Edge Computing, Model Discovery, Internet of Things

I. INTRODUCTION

With the exponential growth of IoT devices and edge computing nodes, billions of event logs are accumulated in a daily basis^[1]. Process mining serves as a crucial bridge connecting data science and business process management^[2], encompassing three key areas: model discovery^[3–8], conformance checking^[9–12], and process enhancement^[13,14]. In addition, process mining also includes subdomains such as process prediction^[15–18] and business process automation^[19]. Among these, model discovery, which extracts process models from event logs, remains one of the most challenging tasks in process mining. Traditional process model discovery techniques rely on centralized storage and processing of complete

event logs. However, when dealing with large-scale event logs generated in distributed edge environments, they face critical challenges such as limited storage and computing capacity and high communication overhead. Existing distributed computing frameworks partially parallelize traditional algorithms, but still rely on partial centralized data aggregation, limiting their ability to meet real-time processing requirements^[20,21]. Similarly, stream-based process mining techniques enable real-time data processing but struggle to achieve an optimal balance between real-time performance and high accuracy^[22,23].

In recent years, EdgeMiner has introduced a novel distributed model discovery technique that constructs a local footprint matrix at data sources (i.e., edge nodes) and efficiently aggregates them at a central node to derive a global process model in [24, 25]. This technique reduces computational and communication overhead at the central node while also enhancing data privacy. The emergence of EdgeMiner provides a new perspective for tackling efficiency, real-time processing, and security challenges in large-scale distributed event data processing. However, EdgeMiner still faces several limitations: (1) low process model discovery efficiency. Despite Most-Frequent-Predecessors(MFPs) and batch processing optimizations, the computational burden remains high, leading to significant delays in complex processes; (2) insufficient time synchronization handling. The technique assumes that event timestamps in the event log are unique and, therefore, failing to effectively handle timestamp conflicts. This can result in incorrect representations of concurrency relations in the local footprint matrix; and (3) algorithmic limitations. While EdgeMiner introduces various distributed model discovery strategies, the currently implementation, edgeAlpha, is based on the Alpha Miner(AM), which cannot handle cyclic process structures, restricting its applicability to complex processes.

To address these challenges, this paper proposes EdgeIM, a novel Inductive Miner-inspired algorithm designed to improve efficiency and applicability in edge-based process model discovery while maintaining high model quality. EdgeIM includes three key steps: First, the cases are sorted and sampled based on timestamps and case ids, and a set filtering strategy is applied to reduce redundant data. Then, each edge node constructs a local directly-follows relation set using timestamps,

This paper is supported by the National Natural Science Foundation of China (No. 62472264 and 52374221), National Science and Technology Major Project(2022ZD0119501), Major Basic Research Program of Shandong Provincial Natural Science Foundation(ZR2024ZD22), 2022 Qingdao West Coast New Area Special Program for Science and Technology(202209), Shandong Provincial Natural Science Foundation(ZR2024QF230).

* Faming Lu is the corresponding author.

while recording the start and end activities of each case. Finally, the central node efficiently aggregates the local information uploaded by edge nodes and recursively decomposes the global directly-follows graph using the Inductive Miner (IM)^[5] technique to discover a Petri net model comparable to those generated by centralized techniques.

The remainder of this paper is organized as follows. Section II reviews related work on process model discovery. Section III introduces the fundamental concepts of edge-based process discovery. Section IV provides an overview of the proposed edge process model discovery technique and details its three key stages. Section V presents experimental evaluations and result analysis. Finally, Section VI concludes the paper.

II. RELATED WORK

Traditional centralized process model discovery techniques rely on centralized storage and processing of complete event logs. While effective small or centralized datasets, these techniques face significant challenges when dealing with large-scale, distributed event data, leading to storage and computational bottlenecks from centralized transmission of sensitive data. With the rise of IoT and big data, event logs have become increasingly massive and distributed. To address the scalability limitations of traditional techniques, researchers have proposed distributed process mining techniques. Some techniques leverage distributed computing frameworks such as MapReduce to parallelize traditional process mining algorithms, improving processing speed and scalability to some extent in [20]. Other techniques perform local data preprocessing at edge nodes, where each node constructs local directly-follows relations or partial process models, which are then aggregated at a central node to derive a global model in [21]. The decentralized technique reduces data transmission and storage overhead at the central node while enhancing real-time performance and scalability. However, these techniques still require frequent data exchanges between local and central nodes, leading to increased communication overhead. Furthermore, they often rely on partial centralized data aggregation, making it difficult to achieve fully real-time processing.

Stream-based process model discovery techniques are designed to handle real-time data stream environments employing techniques such as sliding windows, approximate counting, and dynamic updates to process continuous event streams and update process models online^[22,23]. While these techniques offer high real-time performance, they often involve trade-offs in data noise handling, model stability, and accuracy, making it difficult to achieve both real-time and high precision simultaneously. To address data privacy concerns, some studies have explored techniques like differential privacy to protect sensitive information during process mining in [26]. Although these techniques enhance privacy protection, they often come at the cost of reduced model accuracy or additional computational overhead. Therefore, further improvements are needed to enhance the efficiency, scalability, and data privacy protection of process model discovery techniques, while ensuring the accuracy of the generated process models.

III. BACKGROUND KNOWLEDGE

This section provides an overview of the foundational concepts.

Let S be a set and $\mathcal{P}(S) = \{X \mid X \subseteq S\}$ be the powerset of S . A multi-set over S is defined as a set where elements may occur more than one times, and $B(S)$ represents the set of all multisets over S . $f: X \rightarrow Y$ is a function where $\text{dom}(f)$ represents its domain and $\text{rng}(f) = \{f(x) \mid x \in \text{dom}(f)\}$ represents its range. $g: X \rightarrow Y$ is a partial function if g maps a subset in the domain to a set of unique elements in the range. A finite sequence over set S of length n is a function $\sigma: \{1, 2, \dots, n\} \rightarrow S$. If $\sigma(1) = a_1, \sigma(2) = a_2, \dots, \sigma(n) = a_n$, we write $\sigma = a_1, a_2, \dots, a_n$. S^* denotes the set of all finite sequences over set S . Assume that $u, v \in S^*$ are two finite sequences, $\sigma = u \circ v$ represents its concatenation operation as defined follows: $\sigma: \{1, 2, \dots \mid u \mid + \mid v \mid\} \rightarrow S$, such that $\sigma(i) = u(i)$ for $1 \leq i \leq |u|$, and $\sigma(i) = v(i - |u|)$ for $(|u| + 1) \leq i \leq (|u| + |v|)$.

Given a set X and its subset Q . $\upharpoonright_Q \in X^* \beta Q^*$ is a projection function as defined below: $\langle \rangle \upharpoonright_Q = \langle \rangle$; and for $Q \in X^*$ and $x \in X$:

Definition 1. (Event Log, Trace, Activity)^[27] Let A be a collection of activities, and an event log $L \subseteq B(A^*)$ is a multi-set of traces, where a trace $\sigma \in L$ is a sequence of activities.

For example, $L = [\langle x, a, b, c, e, y \rangle^2, \langle x, a, c, b, e, y \rangle^4, \langle x, b, d, a, f, y \rangle^3, \langle x, b, a, d, f, y \rangle]$ contains ten traces. For $\sigma_1 = \langle x, a, b, c, e, y \rangle \in L$, σ_i represents the i -th trace of L .

$$(\langle x \rangle \circ \sigma) \upharpoonright_Q = \begin{cases} \sigma \upharpoonright_Q, & \text{if } x \notin Q \\ \langle x \rangle \circ (\sigma \upharpoonright_Q), & \text{if } x \in Q \end{cases} \quad (1)$$

Definition 2. (Start Activity, End Activity) Let $\sigma_i = \langle a_1, a_2, \dots, a_n \rangle$ be the i -th trace in event log L . The start activity of σ_i is defined as: $\text{start}(\sigma_i) = a_1$, where $a_1 \in A$. The end activity of σ_i is defined as: $\text{end}(\sigma_i) = a_n$, where $a_n \in A$.

For example, $\sigma_1 = \langle x, a, c, e, y \rangle$, $\text{start}(\sigma_1) = x$, $\text{end}(\sigma_1) = y$.

Definition 3. (Directly-Follows Relation) Given an event log L , the directly-follows relation set R_L is constructed as:

$$R_L = \{ \langle a_k, a_{k+1} \rangle \mid \sigma = \langle a_1, \dots, a_k, a_{k+1}, \dots, a_m \rangle \in L \}$$

where a_k, a_{k+1} are consecutive events in trace σ_i .

The weight function $w: R_L \rightarrow \mathbb{N}^+$ assigns to each relation $(a, b) \in R_L$ its occurrence count:

$$w(a, b) = \sum_{\sigma_i \in L} \sum_{k=1}^{|\sigma_i|-1} \delta(a, \sigma[k]) \cdot \delta(b, \sigma[k+1]) \quad (2)$$

where δ is the Kronecker delta function:

$$\delta(x, y) = \begin{cases} 1 & \text{if } x = y, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

For example, in event log $L = [\langle x, a, b, c, e, y \rangle^2, \langle x, a, c, b, e, y \rangle^2, \langle x, a, b \rangle]$, (a, b) occurs twice, (a, c) occurs 4 times.

Definition 4. (Directly-Follows Graph) A Directly-Follows Graph (DFG) is a directed graph $G = (V, E, w)$, such that (1) V represents the set of activities; (2) E represents the set of directed edges, indicating the “directly-follows” relations between activities; and (3) w represents a weight to each edge, representing the frequency of the corresponding directly-follows relation.

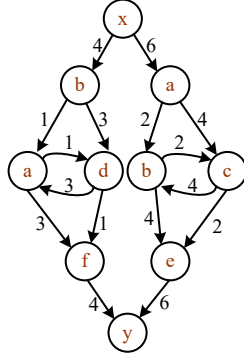


Fig. 1. Directly-follows graph example.

An example of DFG is shown in Fig. 1. The set of activities $V = \{x, a, b, c, d, e, f, y\}$, where x is the start activity and y is the end activity. The weight of the edge $x \rightarrow a$ is 6.

Definition 5. (Petri Nets)^[28] A Petri Net is defined as $PN = (P, T, F, l)$, such that (1) $P \cap T = \emptyset$, $P \cap T \neq \emptyset$ where P denotes the place set and T denotes the transition set; (2) $F \subseteq (P \times T) \cup (T \times P)$ denotes the flow relation between transitions and places; and (3) $l : T \rightarrow \mathcal{A}$ is a transition labeling function where $\tau \in \mathcal{A}$ is a label set and represents an invisible label.

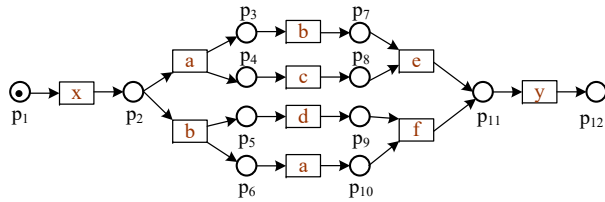


Fig. 2. Petri net example.

Fig. 2 shows an example Petri net of the event log L . The formalization of this Petri net is as follows: $P = \{p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9, p_{10}, p_{11}, p_{12}\}$, $T = \{x, a, b, c, d, e, f, y\}$, $F = \{(p_1, x), (x, p_2), (p_2, a), (p_2, b), (a, p_3), (a, p_4), (p_3, b), (p_4, c), (b, p_7), (c, p_8), (p_7, e), (p_8, e), (e, p_{11}), (b, p_5), (b, p_6), (p_5, d), (p_6, a), (d, p_9), (a, p_{10}), (p_9, f), (f, p_{11}), (p_{11}, y), (y, p_{12})\}$.

IV. EDGE-BASED PROCESS MODEL DISCOVERY

A. An Overview

This section provides an overview of the proposed edge-based process model discovery technique, as shown in Fig. 3. The technique consists of three key stages:

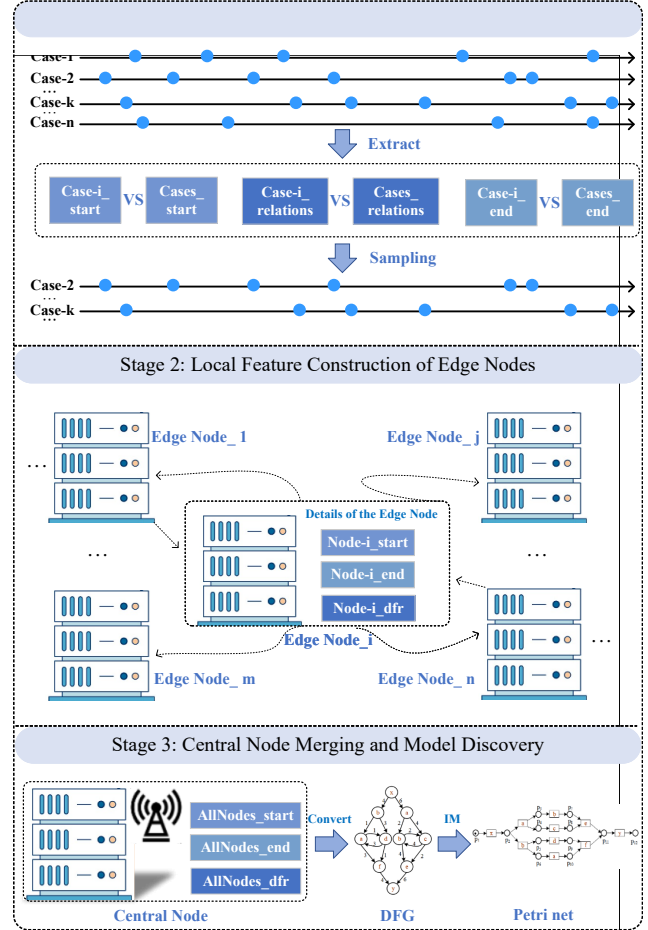


Fig. 3. An Approach Overview.

- **Stage 1: Preprocessing and Feature-Preserving Sampling.** All events in the input cases are sorted in a total order based on the case ID and timestamp to establish a deterministic event sequence. Then, a global feature set that includes start events, end events, and directly-follows relations are constructed. All cases are traversed and only those that expand the global feature set are retained. Next, data filtering is performed by checking whether case features are a subset of the global feature set. Note that this stage ensures that key process structures remain intact while reducing redundancy.
- **Stage 2: Local Feature Construction of Edge Nodes.** Using a hash function, each activity type is mapped to an edge node, enabling distributed allocation of activities across computational nodes. At each node, a local

directly-follows relation set is constructed by ordering activities based on timestamps. The start and end activities of each case are also recorded. When consecutive events share the same timestamp, bidirectional directly-follows relations are added simultaneously to the local relation set, to ensure an accurate representation of concurrency.

- **Stage 3: Central Node Merging and Model Discovery.** The central node aggregates all local features and start/end activity sets from the edge nodes to generate a global directly-follows graph (DFG). Then, the global DFG is recursively decomposed and a Petri net process model is generated.

B. Preprocessing and Feature-Preserving Sampling

The objective of this stage is to efficiently reduce the scale of input data using a dynamic feature coverage strategy. First, the events in the input cases are totally ordered by the lexicographic order of $\langle CaseID, Timestamp \rangle$, eliminating the timestamp uncertainty caused by the distributed collection. Then, a global feature set $A=(S, E, R)$ is established, where S represents the set of start activities, E represents the set of end activities, and R represents the set of directly-follows relations. During case traversal, a case is retained only if its features ($s_i \in S, e_i \in E, r_{ij} \in R$) extend the global feature set; otherwise, it is filtered. This technique ensures zero loss of key process structure features by performing a set inclusion check: $S_i \subseteq S \wedge E_i \subseteq E \wedge R_i \subseteq R$. The proposed algorithm is detailed in Algorithm 1.

Algorithm 1: Feature-Preserving Sampling

Input : $cases_set = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$.
Output: *Filtered D' , Global Features (S, E, R) .*

- 1 **Sort** L by $\langle CaseID, Timestamp \rangle$ to obtain ordered cases $Case_ord$.
- 2 **Initialize** $S \leftarrow \emptyset, E \leftarrow \emptyset, R \leftarrow \emptyset$
- 3 **for each** trace σ in $Case_ord$ **do**
- 4 Extract $s \leftarrow \sigma[0], e \leftarrow \sigma[-1]$,
 $R_{tmp} \leftarrow \{(\sigma[k], \sigma[k+1]) \mid 1 \leq k < |\sigma|\}$
- 5 **if** ($s \notin S$) $\vee$ ($e \notin E$) $\vee$ ($R_{tmp} \not\subseteq R$) **then**
- 6 $D' \leftarrow D' \cup \{\sigma\}$ // Add log to D'
- 7 $S \leftarrow S \cup \{s\}$
- 8 $E \leftarrow E \cup \{e\}$
- 9 $R \leftarrow R \cup R_{tmp}$
- 10 **end**
- 11 **end**
- 12 **return** D', S, E, R

Algorithm 1 employs a set-theoretic coverage strategy where, to ensure that the sampled cases D' fully represent the key features of the original process. The time complexity of the algorithm is primarily determined by two components: sorting and feature checking. The time complexity of the sorting is $O(n * \log n)$, where n is the total number of events. As for the feature checking component, all traces are traversed, and each set operation has a complexity of $O(m^2)$ where m is the

maximum number of activities in a single trace. Thus, the overall complexity of Algorithm 1 is $O(n * m^2)$.

C. Local Feature Construction of Edge Nodes

In a distributed computing framework, a consistent hashing algorithm maps activity types to edge nodes, ensuring balanced workload distribution. Each node independently maintains a local feature triplet (S_i, E_i, R_i) , where S_i is the local start activity set, E_i is the end activity set, and R_i is the weighted direct-follows relations. When consecutive events share the same timestamp, i.e., $t(a_k) = t(a_{k+1})$, both bidirectional relations $\langle a_k, a_{k+1} \rangle$ and $\langle a_{k+1}, a_k \rangle$ in R_i to preserve the potential of concurrent structures. This design ensures that the local Directly-Follows Graph (DFG) can effectively handle timestamp conflicts, so as to maintain process integrity. The algorithm is illustrated in Algorithm 2. The time complexity of the hashing mapping in Algorithm 2 is $O(1)$ per event. The feature construction stage has a time complexity of $O(n)$, where n is the number of events after filtering.

Algorithm 2: Edge-Side Feature Construction

Input : *Filtered D' , Global Features (S, E, R) .*
Output: *Local Features $\{Local_i = (S_i, E_i, R_i)\}$*
 for each n_i

- 1 **for each** node n_i in N **do**
- 2 $S_i \leftarrow \emptyset$
- 3 $E_i \leftarrow \emptyset$
- 4 $R_i \leftarrow \emptyset$
- 5 **end**
- 6 **for each** trace σ in D' **do**
- 7 **for each** event e in σ **do**
- 8 $j \leftarrow \text{hash}(e.\text{activity}) \bmod k$
- 9 **if** e is first event in σ **then**
- 10 $S_j \leftarrow S_j \cup \{e.\text{activity}\}$
- 11 **end**
- 12 **if** e is last event in σ **then**
- 13 $E_j \leftarrow E_j \cup \{e.\text{activity}\}$
- 14 **end**
- 15 **if** previous event e_prev exists **then**
- 16 **if** $e_prev.\text{timestamp} == e.\text{timestamp}$ **then**
- 17 $R_j[\langle e_prev.\text{activity}, e.\text{activity} \rangle] \leftarrow$
 $R_j[\langle e_prev.\text{activity}, e.\text{activity} \rangle] + 1$
- 18 $R_j[\langle e.\text{activity}, e_prev.\text{activity} \rangle] \leftarrow$
 $R_j[\langle e.\text{activity}, e_prev.\text{activity} \rangle] + 1$
- 19 **else**
- 20 $R_j[\langle e_prev.\text{activity}, e.\text{activity} \rangle] \leftarrow$
 $R_j[\langle e_prev.\text{activity}, e.\text{activity} \rangle] + 1$
- 21 **end**
- 22 **end**
- 23 **end**
- 24 **end**
- 25 **return** $\{Local_i = (S_i, E_i, R_i)\}$

D. Central Node Merging and Model Discovery

The central node receives local feature triplets from all edge nodes and constructs the global features using set union operations. This process merges the start activity set, end activity set, and the global Directly-Follows Graph (DFG). Based on this DFG, the algorithm applies the recursive decomposition strategy of the classical Inductive Miner. First, basic patterns, i.e., sequence($\rightarrow$), parallel($\wedge$), choice($\times$), and loop($\odot$) structures, are identified. Then, the activity set is divided into mutually exclusive subsets, and a sub-DFG is generated for each subset. Finally, The sub-nets are merged into a complete process model based on the detected patterns. The whole process is shown in Algorithm 3.

Algorithm 3: Feature-Preserving Sampling

Input : $\{Local_i = (S_i, E_i, R_i)\}$.

Output: Petri Net PN .

```

1  $S \leftarrow \emptyset, E \leftarrow \emptyset, DFG \leftarrow$  empty graph
2 for each  $Local_i$  do
3    $S \leftarrow S \cup S_i$ 
4    $E \leftarrow E \cup E_i$ 
5   foreach  $(a, b) \in R_i$  do
6      $DFG(a, b) \leftarrow DFG(a, b) + R_i(a, b)$ 
7   end
8 end
9 Function  $DFG, S, E$ 
10 if  $|DFG.nodes| = 1$  then
11   return  $CREATELEAFNODE(DFG.nodes[0])$ 
12 end
13  $pattern \leftarrow DETECTBASEPATTERN(DFG)$ 
14    $// \rightarrow, \wedge, \times, \odot$ 
15  $partitions \leftarrow SPLITDFG(DFG, pattern)$ 
16  $subnets \leftarrow \emptyset$ 
17 for each partition  $P \in partitions$  do
18    $subDFG \leftarrow SUBGRAPH(DFG, P)$ 
19    $subnet \leftarrow MINEMODEL(subDFG, S, E)$ 
20    $subnets \leftarrow subnets \cup \{subnet\}$ 
21 end
22  $PN \leftarrow MERGESUBNETS(subnets, pattern)$ 
23 return  $PN$ 
24 return  $PN$ 

```

The global feature aggregation stage of Algorithm 3 has a time complexity of $O(k * m)$, where k is the number of edge nodes and m is the maximum number of local relations per node. The recursive model generation follows Inductive Miner's complexity, which is $O(A^3)$, where A is the number of activities. The primary overhead comes from the recursive partitioning of the DFG and the extraction of subgraphs.

V. EXPERIMENTAL EVALUATION

This section evaluates the effectiveness of the proposed technique by comparing it to state-of-the-art process discovery techniques.

A. Experiment Setup

The efficient edge-based model discovery technique, named as EdgeIM, has been implemented in the PM4Py¹ process mining tool platform. The experiments were conducted on a server with the following specifications: *Intel Xeon Silver 4210R* processor (2.4GHz, 10 cores/20 threads, total 40 cores), 27.5MB L3 cache, supporting VT-x virtualization, and running *Ubuntu 22.04.2 LTS*. Table I provides detailed statistics on nine publicly event logs², including the number of traces (#Trace), the number of events (#Event), the number of activities (#Activity), and trace length information.

B. Model Quality Analysis

This section introduces the F-measure^[29], a widely used quality evaluation metric in process mining. The F-measure is the harmonic mean of fitness^[30] and precision^[31], balancing both aspects to assess overall model quality. Fitness measures how well the process model can accurately reproduce traces recorded in the event log, indicating its ability to regenerate observed behavior. Precision measures the extent to which the process model produces only the traces recorded in the event log, quantifying the portion of behavior that can be replayed in the model but is not observed in the event log. The F-measure is computed using the following formula.

$$F\text{-measure} = \frac{2 \times \text{fitness} \times \text{precision}}{\text{fitness} + \text{precision}} \quad (4)$$

The experimental results for F-measure are presented in Table II. A “-” indicates that the technique either failed to discovery a process model or the generated process model could not be evaluated for the F-measure.

Based on the results in Table II, the following conclusions can be drawn. EdgeIM achieves fitness values comparable to those of the Inductive Miner(IM). The fitness for fightCars log is less than 1 due to the presence of duplicate tasks in the event log, which causes the discovered model to treat these duplicates as the same task. EdgeIM outperforms both Alpha Miner(AM) and EdgeAlpha in terms of fitness, indicating that it fully captures behavioral patterns in the event log while overcoming the structural loss issues encountered by EdgeAlpha due to distributed processing. For complex event logs, e.g., Sepsis and BPI2013Incidents, EdgeIM's precision is about 40% higher than that of the IM. In Sepsis Cases, precision increase from 0.0866 to 0.1213 with EdgeIM, which can be attributed to its recursive discovery approach from the DFG, compared to the IM's divide-and-conquer approach applied to the event log. Differences in fall-through structure handling result in varying placements of invisible transitions, which may influence the precision. Across most datasets, EdgeIM significantly outperforms EdgeAlpha and the AM in precision, demonstrating its ability to handle non-unique event timestamps and mitigate model overfitting.

¹<https://processintelligence.solutions/pm4py>

²<https://data.4tu.nl/search?search=process+mining>

TABLE I
BASIC STATISTICS OF EXPERIMENTAL LOGS

Event Log	#Trace	#Event	#Activity	Trace Length		
				Minimum	Average	Maximum
Sepsis Cases	1050	15214	16	3	14	185
exercise	21	552	20	11	26	47
ETM_Configuration	100	590	7	5	6	6
BPI2013Incidents	7554	65533	13	1	9	123
TSL.anon	17812	83286	40	3	5	47
ICP.anon	582	2956	6	5	5	9
fightCar	300	1467	7	4	5	7
BPI2017OfferLog	12899	58129	8	3	5	5
ALLTrace	2000	18364	10	8	9	12

TABLE II
EXPERIMENTAL RESULTS

Event Log	Alpha Miner			Inductive Miner			EdgeAlpha			EdgeIM		
	fitness	precision	F-measure	fitness	precision	F-measure	fitness	precision	F-measure	fitness	precision	F-measure
Sepsiscases	0.16	0.4272	0.2328	1	0.0866	0.1594	0.16	0.4272	0.2328	1	0.1213	0.2166
exercise	1	0.6458	0.7848	1	0.2876	0.4467	1	0.6458	0.7848	1	0.3008	0.4625
ETMconfig	0.99	0.9918	0.9909	1	0.9918	0.9959	0.99	0.9918	0.9909	1	0.9918	0.9959
BPI13Incid	0.58	0.1489	0.2370	1	0.2672	0.4217	0.17	0.2837	0.2126	1	0.2837	0.4420
TSL.anon	-	-	-	1	0.66	0.7952	-	-	-	1	0.7801	0.8765
ICP.anon	0.62	0.2593	0.3657	1	0.5489	0.7088	0.62	0.2593	0.3657	1	0.443	0.6140
fightCar	0.81	1	0.8950	0.92	0.8525	0.8850	0.81	1	0.8950	0.81	1	0.8950
BPI17Offe	-	-	-	1	0.8246	0.9039	-	-	-	1	0.8246	0.9039
AllTrace	0.9	0.375	0.5294	1	0.9216	0.9592	0.9	0.375	0.5294	1	0.9216	0.9592

EdgeIM's F-measure is comparable to that of the IM and superior to both EdgeAlpha and AM. The slightly lower F-measure for EdgeIM in the Sepsis log is due to the dataset containing multiple start and end activities and loop structures. To ensure model soundness, EdgeIM employs more invisible transitions during model generation. In contrast, EdgeAlpha and the AM exclude invisible transitions, causing some loop nodes to be separated from the overall model, thereby compromising model soundness. These results indicate that EdgeIM effectively discovers sound process models and is capable of handling loop structures.

C. Time Performance Analysis

The runtime of AM and IM is not directly comparable to edge-based model discovery techniques, as they do not involve edge data processing and can directly discover process models from event logs. Therefore, this experiment focuses on comparing the execution time of EdgeIM and EdgeAlpha. To ensure the accuracy and validity of the results, the experiment measured the runtime five times for each dataset and calculated the average execution time. The final time performance results are shown in Table III.

Based on Table III, the following conclusions can be drawn. The runtime of EdgeIM consists of the sampling time (Stage 1) and the discovery time (Stages 2-3). Across all datasets, EdgeIM outperforms EdgeAlpha in total execution time. Although the number of edge nodes after Stage 1 filtering remains equal to the initial number, removing redundant cases significantly reduces the mutual accesses among edge nodes. This reduction in inter-node communication overhead

improves the efficiency of edge-based model discovery. In the exercise log, EdgeIM takes longer than EdgeAlpha. This is because the dataset is small, contains no redundant cases, and has a higher proportion of fixed overhead for preprocessing and sampling. However, this does not diminish EdgeIM's advantage in large-scale, complex IoT scenarios. These results confirm that EdgeIM is better suited for distributed, large-scale, and high-complexity process discovery.

VI. CONCLUSION

To address the challenges of existing process model discovery techniques for IoT scenarios, this paper proposes EdgeIM. By integrating edge computing with the IM algorithm, EdgeIM enables efficient and privacy-preserving model discovery in IoT environments. Compared to existing edge-based model discovery techniques, EdgeIM effectively handles complex process patterns and improves model reliability through dynamic handling of timestamp conflicts. Moreover, compared to traditional model discovery techniques, EdgeIM maintains high model accuracy while offering superior scalability, making it well-suited for large-scale IoT deployments.

Several areas warrant further exploration to enhance edge-based process model discovery: Effective strategies for dynamically merging and deploying edge nodes based on the structural features of process models should be explored. Similarly, allocation strategies for edge nodes to further optimize resource utilization and communication efficiency are urgent needed. In addition, cross-modal feature alignment and joint modeling techniques can be used to further improve the accuracy of discovered complex process models.

TABLE III
EXPERIMENTAL RESULTS OF DISCOVERY PERFORMANCE

	EdgeAlpha	EdgeIM		
Event Log	time	sum time	sampling time	discovery time
Sepsis Cases	1957.3217	174.2978	4.732	169.5658
exercise	86.5827	90.6906	2.125	88.5656
ETM_Configuration	85.4518	24.5927	4.214	20.3787
BPI2013Incidents	7574.9726	765.4889	5.299	760.1899
TSL.anon	-	580.5401	32.325	548.2151
ICP.anon	1200.6847	25.7998	6.877	18.9228
fightCar	125.6428	21.4605	6.215	15.2455
BPI2017OfferLog	-	681.4961	36.245	645.2511
AllTrace	2013.5815	21.0402	3.851	17.1892

REFERENCES

- [1] Y. Bertrand, J. De Weerd, and E. Serral, "A bridging model for process mining and iot," in *Process Mining Workshops*. Cham: Springer International Publishing, 2022, pp. 98–110.
- [2] W. M. van der Aalst, *Process mining : discovery, conformance and enhancement of business processes*. Germany: Springer, 2011.
- [3] W. Van der Aalst, T. Weijters, and L. Maruster, "Workflow mining: Discovering process models from event logs," *IEEE transactions on knowledge and data engineering*, vol. 16, no. 9, pp. 1128–1142, 2004.
- [4] A. J. Weijters, W. M. van Der Aalst, and A. A. De Medeiros, "Process mining with the heuristics miner algorithm," 2006.
- [5] S. J. Leemans, D. Fahland, and W. M. Van Der Aalst, "Discovering block-structured process models from event logs-a constructive approach," in *International conference on applications and theory of Petri nets and concurrency*. Springer, 2013, pp. 311–329.
- [6] S. J. J. Leemans, D. Fahland, and W. M. van der Aalst, "Scalable process discovery and conformance checking," *Software and Systems Modeling*, vol. 17, pp. 599 – 631, 2018.
- [7] A. Augusto, R. Conforti, M. Dumas, M. La Rosa, and A. Polyvyanyy, "Split miner: automated discovery of accurate and simple business process models from event logs," *Knowledge and Information Systems*, vol. 59, pp. 251–284, 2019.
- [8] H. Nguyen, M. Dumas, A. H. ter Hofstede, M. La Rosa, and F. M. Maggi, "Stage-based discovery of business process models from event logs," *Information Systems*, vol. 84, pp. 214–237, 2019.
- [9] A. S. M. W. Josep Carmona, Boudewijn van Dongen, *Conformance Checking*. Cham: Springer International Publishing, 2018.
- [10] M. Bauer, H. Van der Aa, and M. Weidlich, "Estimating process conformance by trace sampling and result approximation," in *Business Process Management: 17th International Conference, BPM 2019, Vienna, Austria, September 1–6, 2019, Proceedings 17*. Springer, 2019, pp. 179–197.
- [11] M. Fani Sani, S. J. van Zelst, and W. M. van der Aalst, "Conformance checking approximation using subset selection and edit distance," in *Advanced Information Systems Engineering: 32nd International Conference, CAiSE 2020, Grenoble, France, June 8–12, 2020, Proceedings 32*. Springer, 2020, pp. 234–251.
- [12] M. Bauer, H. van der Aa, and M. Weidlich, "Sampling and approximation techniques for efficient process conformance checking," *Information Systems*, vol. 104, p. 101666, 2022.
- [13] S. J. van Zelst, B. F. van Dongen, and W. M. van der Aalst, "Online discovery of cooperative structures in business processes," in *On the Move to Meaningful Internet Systems: OTM 2016 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2016, Rhodes, Greece, October 24–28, 2016, Proceedings*. Springer, 2016, pp. 210–228.
- [14] D. Fahland and W. M. Van Der Aalst, "Model repair—aligning process models to reality," *Information Systems*, vol. 47, pp. 220–243, 2015.
- [15] N. Guo, C. Liu, C. Li, Q. Zeng, C. Ouyang, Q. Liu, and X. Lu, "Explainable and effective process remaining time prediction using feature-informed cascade prediction model," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 949–962, 2024.
- [16] M. Zou, Q. Zeng, C. Liu, R. Cao, S. Chen, and Z. Zhao, "Prediction of remaining execution time of business processes with multiperson collaboration in assembly line production," *IEEE Transactions on Computational Social Systems*, 2024.
- [17] G. Na, L. Cong, L. Caihong, L. Wenjuan, W. Lei, and Z. Qingtian, "Predictive monitoring method of wind turbine overhauls process based on feature selection," *Computer Integrated Manufacturing System*, vol. 30, no. 8, p. 2745, 2024.
- [18] M. Zou, Q. Zeng, H. Duan, W. Ni, and S. Chen, "Tv-alp: A log dataset of television assembly line production under multi-person collaboration for process mining research," *Applied Intelligence*, vol. 54, no. 5, pp. 3990–4011, 2024.
- [19] X. L. . W. M. P. v. d. A. Junxiong Gao, Sebastiaan J. van Zelst, "Automated robotic process automation: A self-learning approach," in *OTM Confederated International Conferences: On the Move to Meaningful Internet Systems*. Springer-Verlag, 2019, pp. 95–112.
- [20] J. Evermann and G. Assadipour, "Big data meets process mining: Implementing the alpha algorithm with map-reduce," in *Proceedings of the 29th Annual ACM Symposium on Applied Computing*, 2014, pp. 1414–1416.
- [21] R. Gatta, M. Vallati, J. Lenkiewicz, C. Masciocchi, F. Cellini, L. Boldrini, C. Fernandez Llatas, V. Valentini, and A. Damiani, "On the feasibility of distributed process mining in healthcare," in *Computational Science—ICCS 2019: 19th International Conference, Faro, Portugal, June 12–14, 2019, Proceedings, Part V 19*. Springer, 2019, pp. 445–452.
- [22] A. Burattin, A. Sperduti, and W. M. van der Aalst, "Heuristics miners for streaming event data," *arXiv preprint arXiv:1212.6383*, 2012.
- [23] —, "Control-flow discovery from event streams," in *2014 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2014, pp. 2420–2427.
- [24] J. Andersen, P. Rathje, and O. Landsiedel, "Edgealpha: Distributed process discovery at the data sources," 2024. [Online]. Available: <https://arxiv.org/abs/2405.03426v3>
- [25] J. Andersen, P. Rathje, C. Imenkamp, A. Koschmider, and O. Landsiedel, "Edgeminer: Distributed process mining at the data sources," in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, 2025, pp. 705–713.
- [26] F. Mannhardt, A. Koschmider, N. Baracaldo, M. Weidlich, and J. Michael, "Privacy-preserving process mining: Differential privacy for event logs," *Business & Information Systems Engineering*, vol. 61, pp. 595–614, 2019.
- [27] X. Su, C. Liu, S. Zhang, and Q. Zeng, "Sampling business process event logs with guarantees," *Concurrency and Computation: Practice and Experience*, vol. 36, no. 13, p. e8077, 2024.
- [28] C. Liu, Y. Wang, L. Wen, J. Cheng, L. Cheng, and Q. Zeng, "Discovering hierarchical multi-instance business processes from event logs," *IEEE Transactions on Services Computing*, vol. 17, pp. 142–155, 2024.
- [29] J. De Weerd, M. De Backer, J. Vanthienen, and B. Baesens, "A robust f-measure for evaluating discovered process models," in *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*. IEEE, 2011, pp. 148–155.
- [30] A. Adriansyah, B. F. van Dongen, and W. M. van der Aalst, "Conformance checking using cost-based fitness analysis," in *2011 IEEE 15th international enterprise distributed object computing conference*. IEEE, 2011, pp. 55–64.
- [31] A. Adriansyah, J. Munoz-Gama, J. Carmona, B. F. van Dongen, and W. M. van der Aalst, "Alignment based precision checking," in *Business Process Management Workshops: BPM 2012 International Workshops, Tallinn, Estonia, September 3, 2012. Revised Papers 10*. Springer, 2013, pp. 137–149.